Basic networking commands in Linux and windows OS

Aim:

To study and execute basic networking commands in windows and Linux as per network config, testing connectivity, checking route information, monitoring network status and troubleshoot network related issues.

Commands:

1) IP Config: Display network configuration

* Windows commands: ip config
* Linux Command: ip add (or) ip config

2) PING: Test connectivity to a remote host

* Windows command - ping google.com
* LINUX - Command - ping google.com

3) Trace Route: Display the path packets take to reach a destination.

* Windows Command - tracert google.com
* Linux Command - traceroute google.com

4) nslookup: Resolves domain names to IP addresses.

* Windows Command - nslookup google.com
* Linux Command - nslookup google.com

5) NETSTAT : Displays active network connections and listening ports.

Windows command - netstat / netstat -a

Linux command - netstat -a / ss -a

6) ARP : Displays the ARP cache

Windows Commands - arp -a

Linux commands - arp -a / ip neigh

7) HOSTNAME : Displays the system hostname

Windows Commands - host name

Linux Commands - host name

8) route : Displays or modifies the routing table.

Windows commands - route print

Linux Commands - ip route

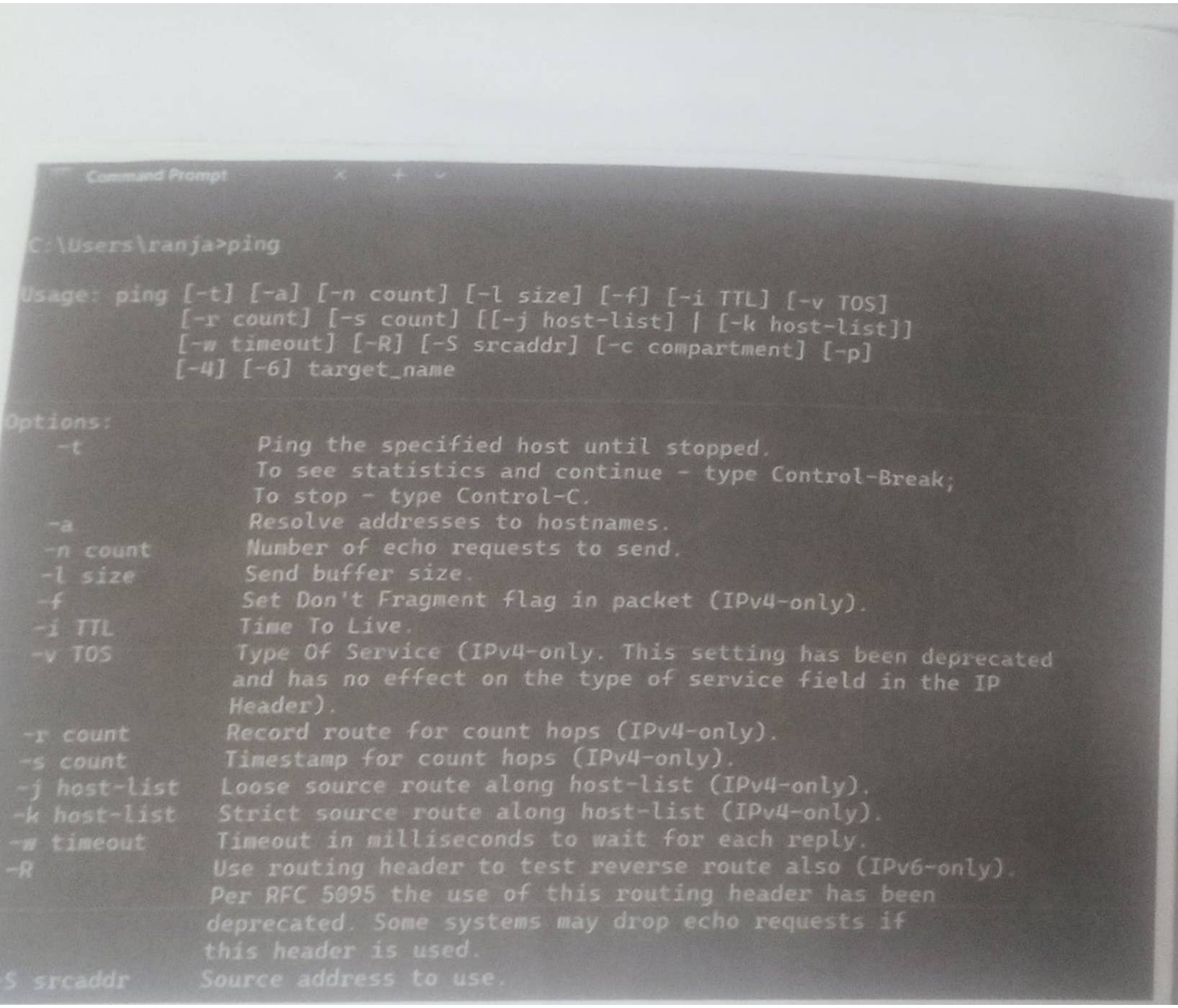
9) MAC Address : Displays MAC address of the computer.

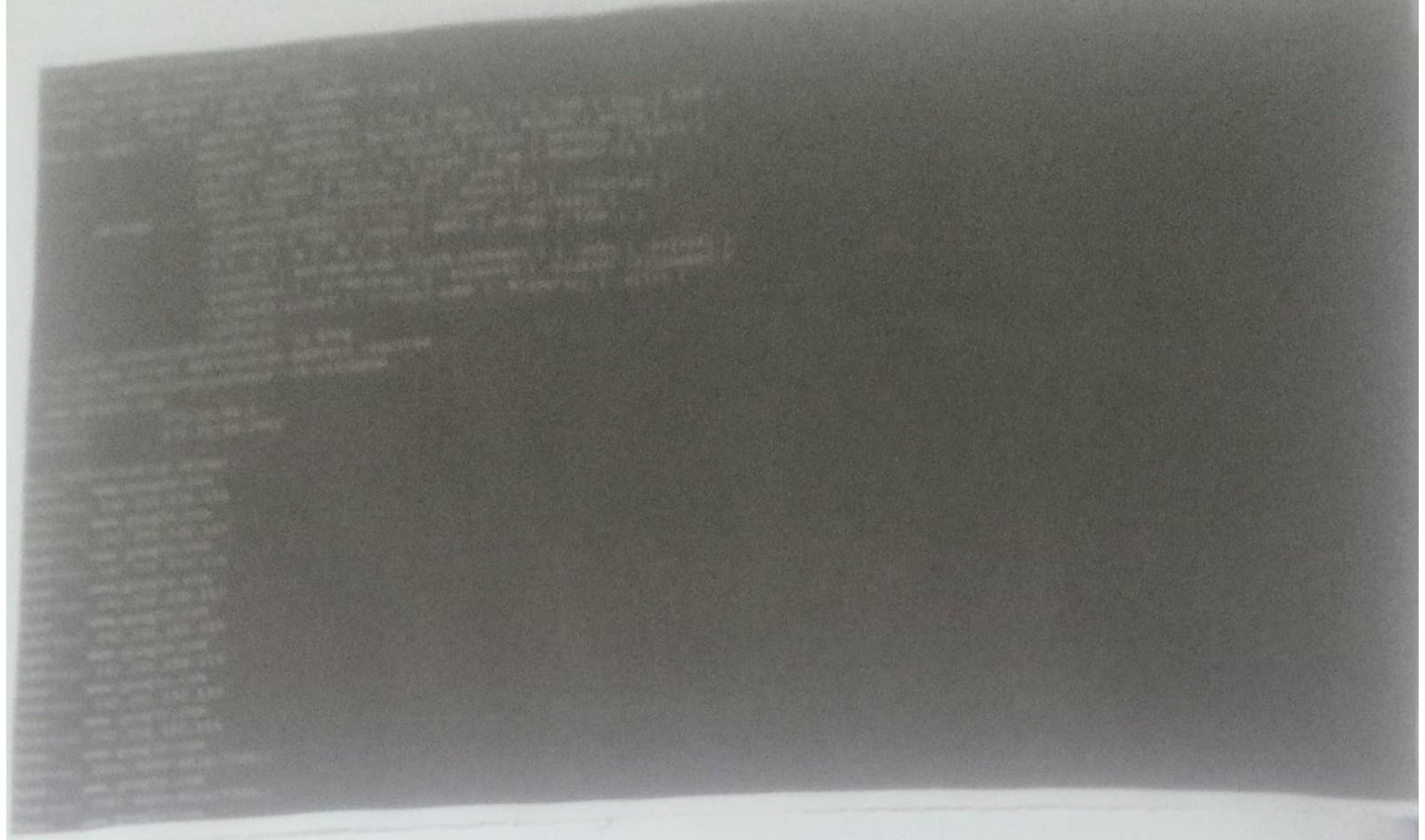
Windows Command - getmac

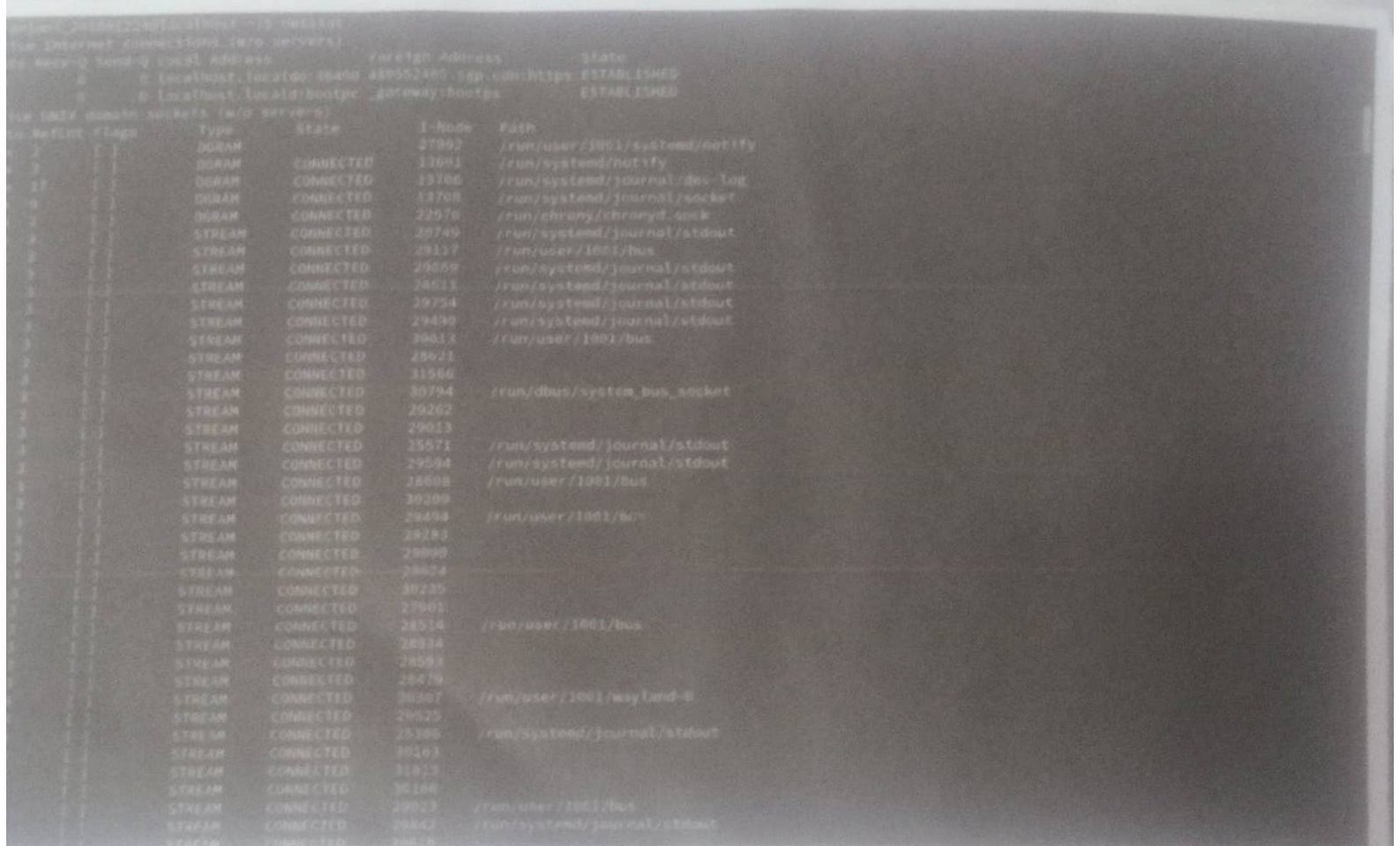
Linux command - ip link

Result :

Thus basic networking commands are executed in both windows CMD and linux terminal successfully.


Command Prompt
C:\Users\ranja>ping
Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
[-r count] [-s count] [[-j host-list] | [-k host-list]]
[-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
[-4] [-6] target_name
Options:
-t Ping the specified host until stopped.
To see statistics and continue - type Control-Break;
To stop - type Control-C.
-a Resolve addresses to hostnames.
-n count Number of echo requests to send.
-l size Send buffer size.
-f Set Don't Fragment flag in packet (IPv4-only).
-i TTL Time To Live.
-v TOS Type Of Service (IPv4-only. This setting has been deprecated
and has no effect on the type of service field in the IP
Header).
-r count Record route for count hops (IPv4-only).
-s count Timestamp for count hops (IPv4-only).
-j host-list Loose source route along host-list (IPv4-only).
-k host-list Strict source route along host-list (IPv4-only).
-w timeout Timeout in milliseconds to wait for each reply.
-R Use routing header to test reverse route also (IPv6-only).
Per RFC 5095 the use of this routing header has been
deprecated. Some systems may drop echo requests if
this header is used.
-S srcaddr Source address to use.





Aim:

To study of different types of network cables

a) Understand different types of network cable

Different type of cables used for networking are

1. Unshielded Twisted Pair (UTP) cable
2. Shielded Twisted Pair (STP) cable
3. Coaxial cable.
4. Fibre optic cable.

Cable type	Category	Maximum Data transmission	Advantages/ Disadvantage	Application/ Use
UTP	category 3 category 5 category 5e	10 bps up to 100 mbps 1 Gbps	Advantage * cheaper in cost Disadvantage *More prone to EMI	10 Base-T Ethernet Fast Ethernet, Gigabit Ethernet
STP	category 6, 6a	10 Gbps	Advantage * Shielded * Faster than UTP Disadvantage * Expensive	Gigabit Ethernet 10G Ethernet (55m) widely used in data centers
STP	category 7	10 Gbps		

Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	* High bandwidth * Immune to interference * Versatile * Limited distance * Cost	
Fibre optics cable	Single mode Multimode	100 Gbps	* High speed * High security * Expensive	

b) Make your own Ethernet cross-over cable

Straight cable

Tools and parts needed.

* Ethernet cabling. CAT5e is certified for gigabit support, but CAT5 cabling works as just over shorter distance.

* A crimping tool. This is an all-in-one tool shaped to push down the pins in the and strip and cut the sheilding off the cables.

* Two RJ45 plugs.

* optional two plug shields.

Step 1: To start construction of the device, begin by threading shields over the cable.

Step 2: Next, strip approximately 1.5 cm of cable shielding from both ends. The crimping tool has a around area to complete this task.

Step 3: After, you will need to untangle the wires; three should be four "twisted pairs." Referencing back to sheet, arrange them from top to bottom. One end should be in arrangement A and the other in B.

Step 4: Once the order is correct, bunch them together in a line, and if there are any that stick out farther than others.

Step 5: Next, push the cable right in. The notch at the end of the plug needs to be just over the cable shielding, and if it isn't, that means that you stripped off too much shielding.

Step 6: After the wires are securely sitting inside the plug, insert into the crimping tool and push down.

Step 7: Lastly, repeat for the other end using diagrams.

Learning and Assignment of IP Address

Aim:

To understand the IP addressing scheme and manually assign static IP address to computers in a network.

Requirements:

Two or more PCs, Network switch/hub, Ethernet cables (RJ45), Windows/Linux OS

Theory / Background:

An IP (Internet protocol) address is a unique numerical label assigned to each device connected to a computer network. IPV_4 addresses are 32-bit numbers written in dotted-decimal notation (192.16 .10). IPV_6 uses 128-bit addresses.

classes of IPV_4 : class A (1.0.0.0-126.255.255.255), class B (128.0.0.0-191.255.255.255), class C (192.0.0. 223.255.255.255).

Subnet mask determines the network and host portions. For class C : 255.255.255.0. Default gate is the router's IP. DNS server translates domain names to IPs.

Manual (static) IP assignment is used for serve and network devices requiring a permanent addresses. DHCP assigns IPs dynamically.

Procedure :

1. In windows : open network and sharing center → change adapter setting → right click Local Area connection → properties → IPV4 → set IP manually

2. Enter IP address : 192.168.1.10, subnet mask : 255.255.255.0, default gateway : 192.168.1.1

3. Enter DNS Server : 8.8.8.8 (Google DNS), click ok

4. In Linux : use command ' sudo ip addr add 192.168.1.10/24 dev eth0 '.

5. To set gateway in Linux : ' sudo ip route add default via 192.168.1.1 '.

6. for persistent configuration in Linux, edit /etc/network/interfaces or use nmcli.

7. Verify the configuration using ' ipconfig ' or ' ip a '.

8. ping the default gateway to confirm network connectivity.

9. ping another computer on the same subnet to verify communication.

commands / code :

sudo ip addr add 192.168.1.10/24 dev eth0

sudo ip route add default via 192.168.1.1

ip a show eth0

ping 192.168.1.1

nmcli con mod ' Wired connection 1 ' ipv4.addresses 192.168.1.10/24

nmcli con mod ' Wired connection 1 ' ipv4.gateway 192.168.1.1

Result :

Static IP addresses were successfully assigned computers. Connectivity was verified using ping commands b/w hosts on the same subnet.

Exp: no: 4

Date: 11/9/24

PACKET SNIFFER USING RAW SOCKETS

Aim: To implement a packet sniffer using raw socket in Python that captures live network traffic and extracts Ethernet, IP, and transport-layer (TCP/UDP/ICMP) headers.

procedure/steps:

step 1: Set up the lab environment

step 2: Understand the packet structure you'll parse.

step 3: Create the Raw socket

step 4: Parse the Ethernet Header

step 5: Parse the IP Header

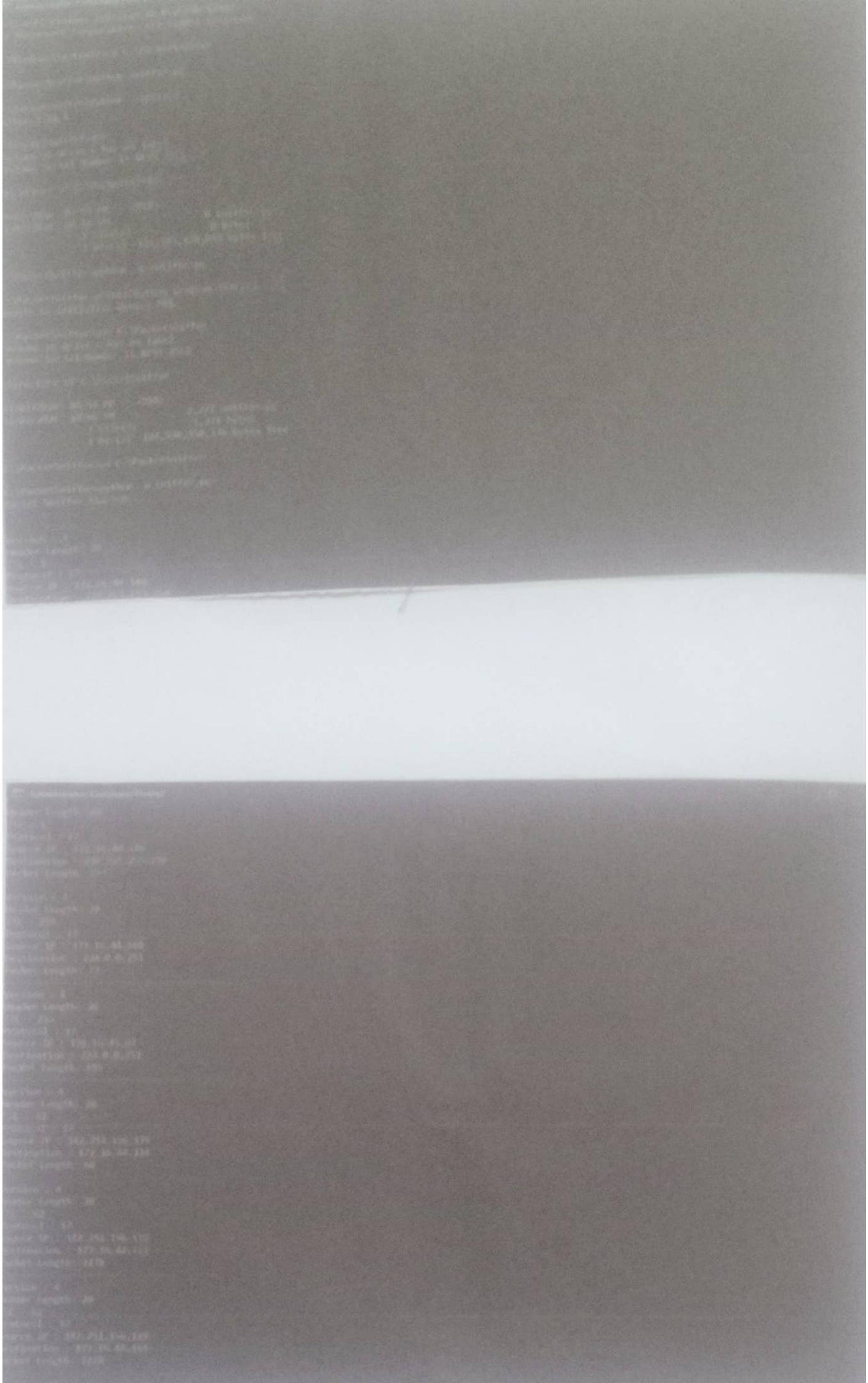
step 6: Parse TCP/UDP/ICMP Headers

step 7: Write the main sniffing loop

step 8: Run the sniffer

Python program:

```
import socket
import struct
sniffer = socket.socket(socket.AF_INET,
                        socket.SOCK_RAW, socket.IPPROTO_IP
host = socket.gethostbyname(socket.gethostname())
sniffer.bind((host, 0))
sniffer.setsockopt(socket.IPPROTO_IP, socket.
                   IP_HDRINCL, 1)
sniffer.ioctl(socket.SIO_RCVALL, socket.
                   RCVALL_ON)
print("packet sniffer started...\n")
```



```
try:
    while True:
        raw_data, addr = s.recvfrom(65535)
        ip_header = raw_data[0:20]
        ipv = struct.unpack('!BBHHHBBH4s4s',
                            ip_header)

        version = ipv[0] >> 4
        ihl = ipv[0] & 0xF

        ttl = ipv[5]
        protocol = ipv[6]
        source = socket.inet_ntoa(ipv[8])
        destination = socket.inet_ntoa(ipv[9])
        print("-----------------")
        print("version :", version)
        print("Header Length :", ihl*4)
        print("TTL :", ttl)
        print("Protocol :", protocol)
        print("source IP :", source)
        print("Destination :", destination)
        print("Packet Length :", len(raw_data))
except KeyboardInterrupt:
    s.ioctl(socket.SIO_RCVALL,
            socket.RCVALL_OFF)

    print("\nStopped")
```

Result:

Thus the packet sniffing program using RAW sockets in python was successfully implemented and executed. The program successfully captured network packets & displayed the IP version, header length, TTL, protocol, source IP address, destination IP address, and packet length

Ex.No: 9 Date: 09/05/25

DEVELOP A CUSTOMIZED PING COMMAND TO TEST THE SERVER CONNECTIVITY

Aim: To develop a customized ping command in python that checks the connectivity with a client and a server by sending ICMP echo Request packets and displaying the Response time & status.

Requirements:

Hardware: Computer/laptop, Internet connection

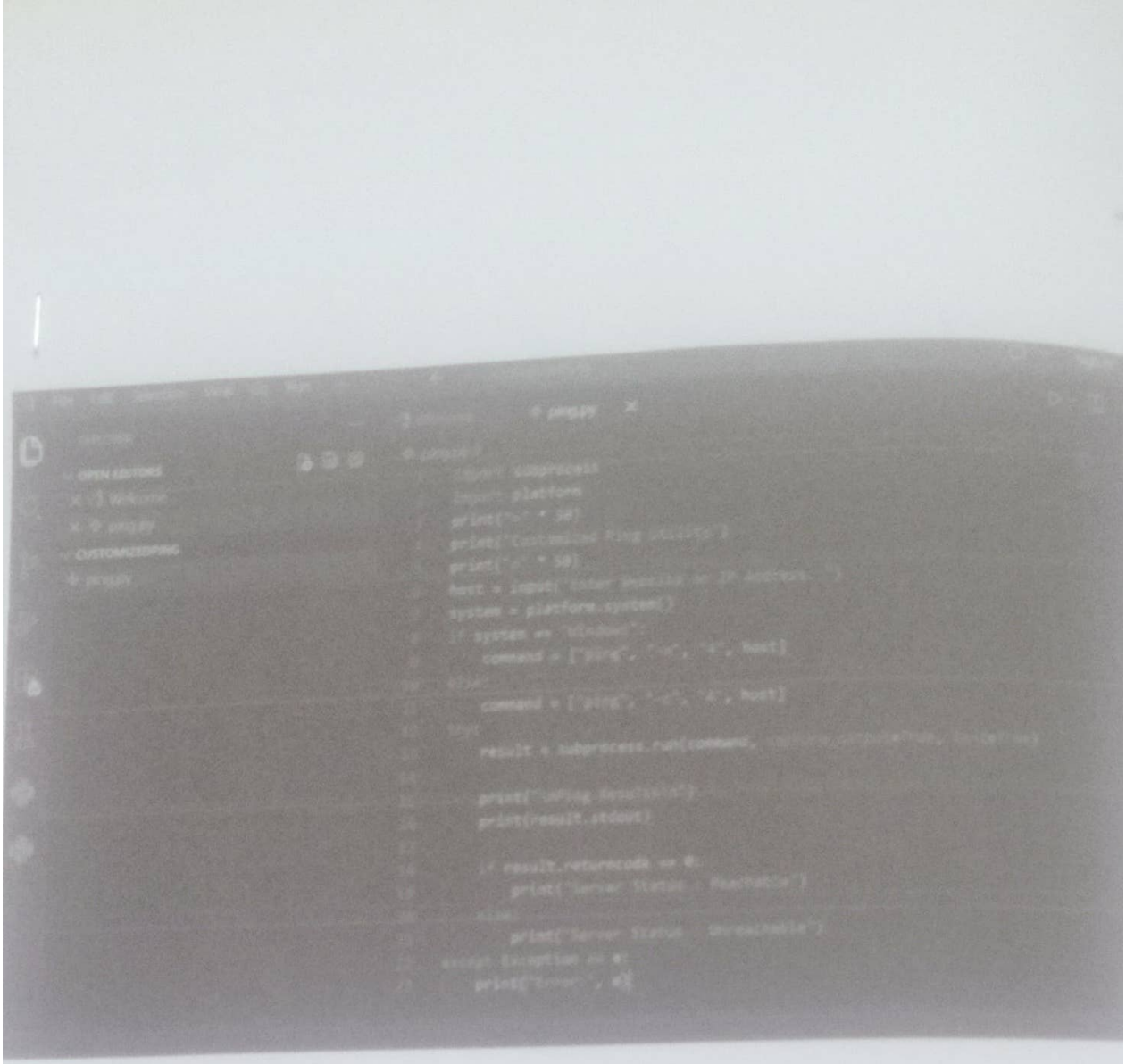
Software: Windows 10/11, python / VS code / Terminal

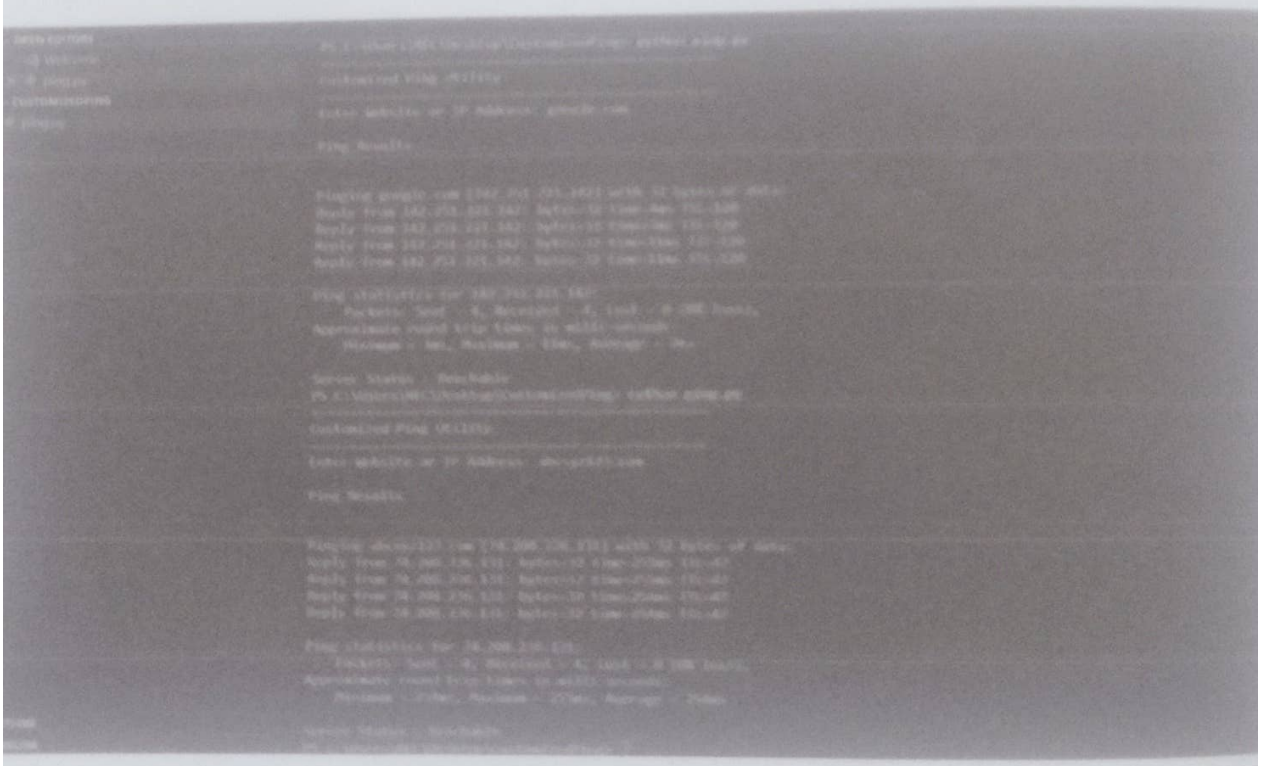
Working procedures:

⇒ Install python & vs code
⇒ create a python file named ping.py
⇒ Import the required modules.
⇒ Accept the hostname or IP address from user
⇒ Detect the OS
⇒ using ping -n on windows or ping -c on linux.
⇒ Execute the ping command using subprocess module.
⇒ Display the ping output, including reply time & packet statistics
⇒ check the return code to determine whether the server is reachable.
⇒ Display reachable or unreachable based on result.

Result:

Thus the customized ping command was successfully developed in python to test server connectivity & display the reply, packet statistics and Reachable / unreachable status.





Building an Anonymous FTP Scanner using Python

Aim:

To develop a python based anonymous FTP scanner using the built in ftplib module to determine whether an FTP server permits anonymous access.

Requirements:

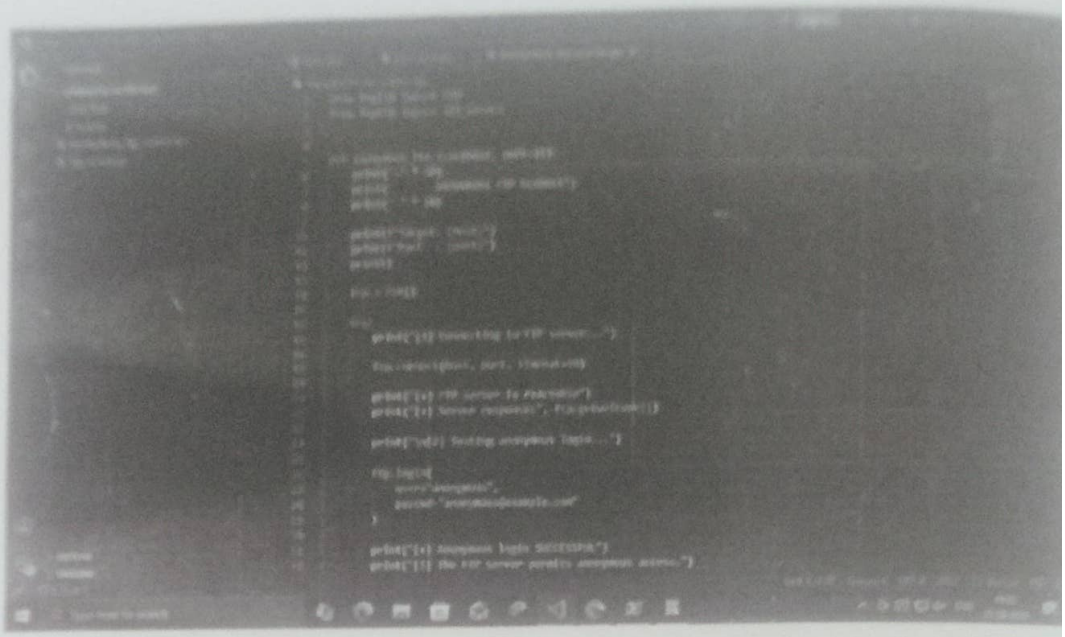
windows/Linux computer, python 2.x, python's built in ftplib module.

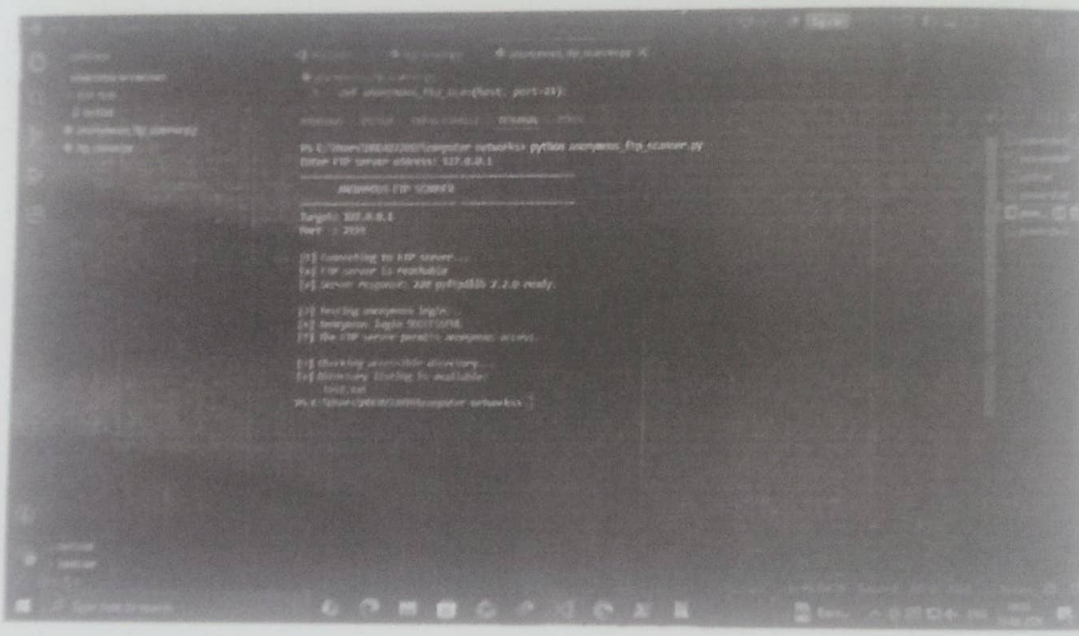
Working procedure:

1) Check python installation
2) open command prompt in windows or terminal in Linux.
3) create the python file named anonymous FTP-scanner.py.
4) Import the ftplib module
5) FTP provides function for connecting & communicating with an FTP server.
6) write the scanner program
7) Run the program

Result:

The python program was executed successfully using the built in ftplib module. the FTP server was connected & for anonymous login.





Ex: NO : 7 develop a simple Calculator using

Date : 2024/9/26 XML-RPC

Aim :

To develop a simple calculator using the XML-RPC protocol in python to perform basic arithmetic operations between a client and server.

Requirements :

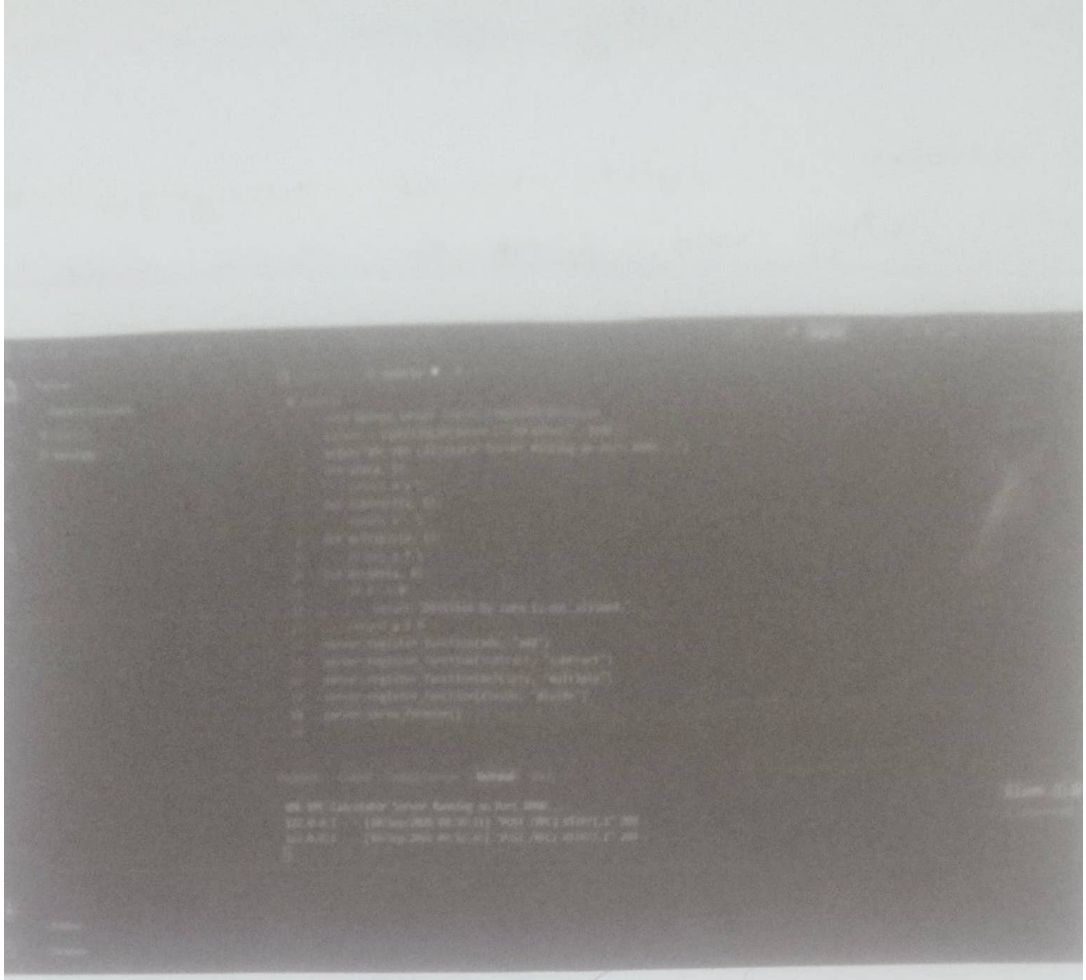
Laptop/Desktop, python, window, VS code

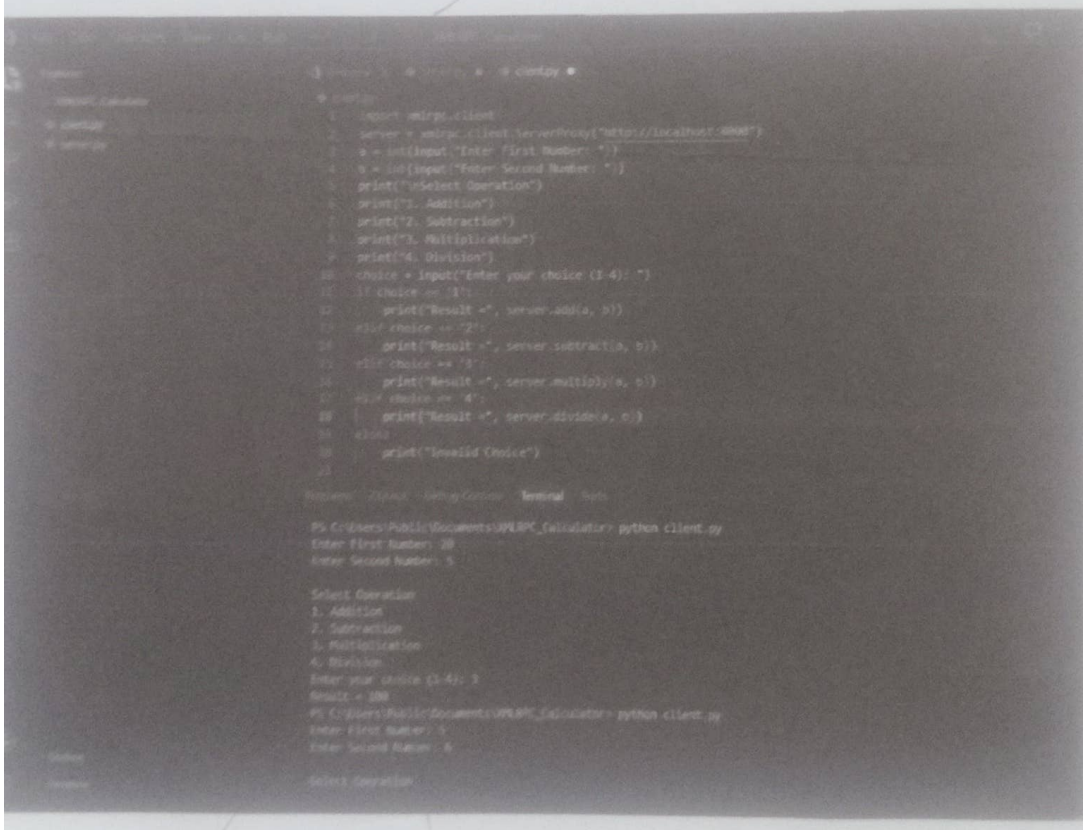
procedure :

1) Install python
2) Verify the python installation
3) create a project folder.
4) Write a server program
   server.py
5) Write a client program
   client.py
6) Start the XML-RPC server
7) Run the client program
8) Enter the input
9) observer the output.

Result :

A simple-Calculator using the XML-RPC protocol in python to perform basic arithmetic operations b/w client and server is developed successfully.





Date : 11/09/25

Aim :

To implement TCP client - server communication using python socket programming

Requirements :

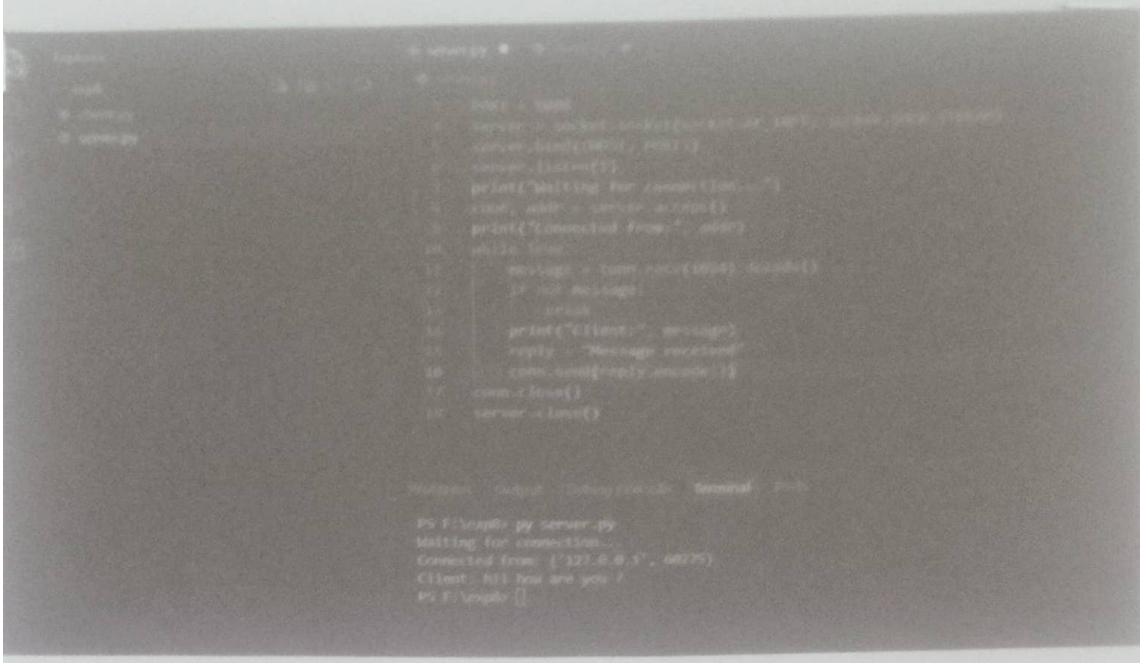
Python 3.x, vs code / IDLE, computer

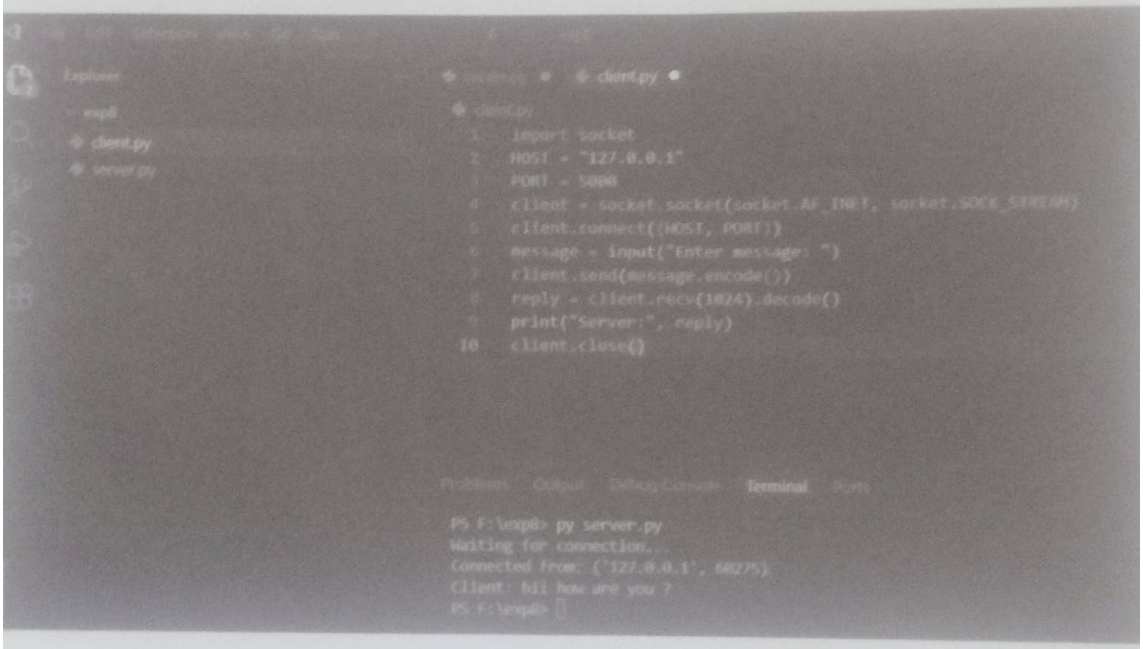
Procedure :

1. Create server.py and client.py
2. Create a TCP socket.
3. Bind the server to port 5000
4. Start the server and accept the client connection.
5. Send and receive messages.
6. Close the connection

Result :

TCP client - server communication was successfully implemented using python.


server.py
PORT = 5000
server.bind((HOST, PORT))
server.listen(5)
print("Waiting for connection...")
print("Connected from:", addr)
message = conn.recv(1024).decode()
print("Client:", message)
reply = "Message received"
conn.send(reply.encode())
conn.close()
server.close()
Terminal
PS F:\exp6> py server.py
Waiting for connection...
Connected from: ('127.0.0.1', 60275)
Client: hii how are you ?
PS F:\exp6>


Explorer
exp6
client.py
server.py
client.py
import socket
HOST = "127.0.0.1"
PORT = 5000
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))
message = input("Enter message: ")
client.send(message.encode())
reply = client.recv(1024).decode()
print("Server:", reply)
client.close()
Problems
Output
Debug Console
Terminal
Ports
PS F:\exp6> py server.py
Waiting for connection...
Connected from: ('127.0.0.1', 60275)
Client: hii how are you ?
PS F:\exp6>

Ex: NO : 4 Design a simple topology & configure one
Date : 24/09/26 Router, two switches & PCs using cisco PT

Aim :

To design and configure a simple network topology consisting of one router, two switches, and four PCs using Cisco packet Tracer and verify connectivity b/w all devices.

procedure :

1. open Cisco Packet Tracer and wait for the workspace to load.

2. Add one router (Cisco 1941 or 2911), two cisco 2960 switches, and four PCs to the workspace

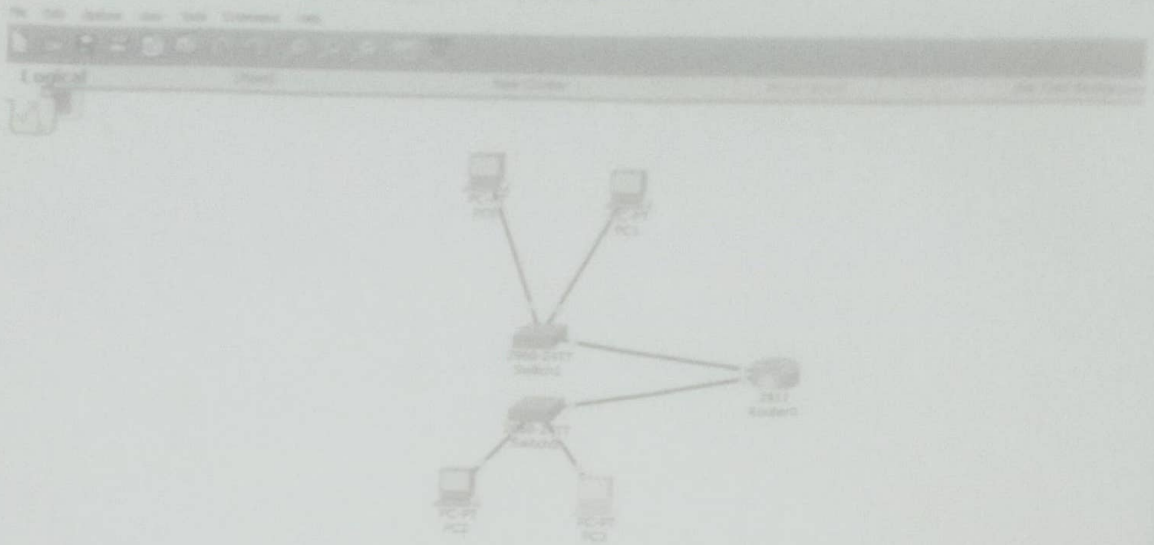
3. Connect the router to the two switches & connect two PCs to each switch using copper straight-Through cables.

4. Configure the routers GigabitEthernet 0/1 Interface with IP Address 192.168.1.1 and subnet mask 255.255.255.0.

5. Configure the routers Gigabit 0/1 Interface with IP Address 192.168.2.1 and subnet mask 255.255.255.0.

6. Enable both router Interfaces using the no shutdown command and save the configuration.

7. Configure the IP Address, subnet masks, and default gateways of all four PCs according to the given network topology.



Physical Config CLI Attributes

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface gigabitEthernet0/0
Router(config-if)#ip address 192.168.1.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up
exit
Router(config)#interface gigabitEthernet0/1
Router(config-if)#ip address 192.168.2.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up
exit
Router(config)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console
copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
Router#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.1.1     YES manual up                    up
GigabitEthernet0/1     192.168.2.1     YES manual up                    up
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
Router#no shutdown
       ^
% Invalid input detected at '^' marker.

Router#
```

Copy Paste

8. Open the command prompt of both PC and use the ping command to test connectivity b/w the devices.

9. Verify that the PC can communicate within their network and across the two networks.

Result:

Thus a simple network topology consisting of one router, two switches, and four PC, was successfully designed and configured using Cisco Packet Tracer. Connectivity b/w the devices was verified using the ping command.

Command Prompt

```
Packet Tracer PC Command Line 1.0
PC>ping 192.168.1.3

Pinging 192.168.1.3 with 32 bytes of data:

Reply from 192.168.1.3: bytes=32 time=38ms TTL=128
Reply from 192.168.1.3: bytes=32 time=0ms TTL=128
Reply from 192.168.1.3: bytes=32 time=0ms TTL=128
Reply from 192.168.1.3: bytes=32 time=0ms TTL=128

Ping statistics for 192.168.1.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 38ms, Average = 9ms

PC>ping 192.168.2.2

Pinging 192.168.2.2 with 32 bytes of data:

Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.

Ping statistics for 192.168.2.2:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

Command Prompt

```
Packet Tracer PC Command Line 1.0
PC>ping 192.168.2.3

Pinging 192.168.2.3 with 32 bytes of data:

Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.

Ping statistics for 192.168.2.3:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

Command Prompt

```
PC>ping 192.168.2.3

Pinging 192.168.2.3 with 32 bytes of data:

Reply from 192.168.2.3: bytes=32 time=21ms TTL=128
Reply from 192.168.2.3: bytes=32 time=1ms TTL=128
Reply from 192.168.2.3: bytes=32 time=3ms TTL=128
Reply from 192.168.2.3: bytes=32 time=1ms TTL=128

Ping statistics for 192.168.2.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 21ms, Average = 6ms
```

Ex: No: 10 Examine network Address Translation

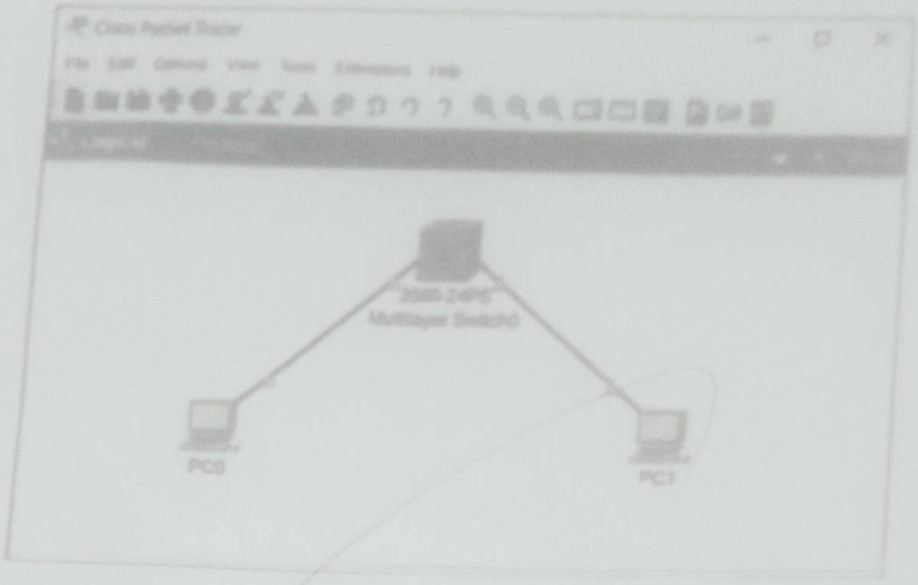
Date: 24/9/24 using cisco packet Tracer

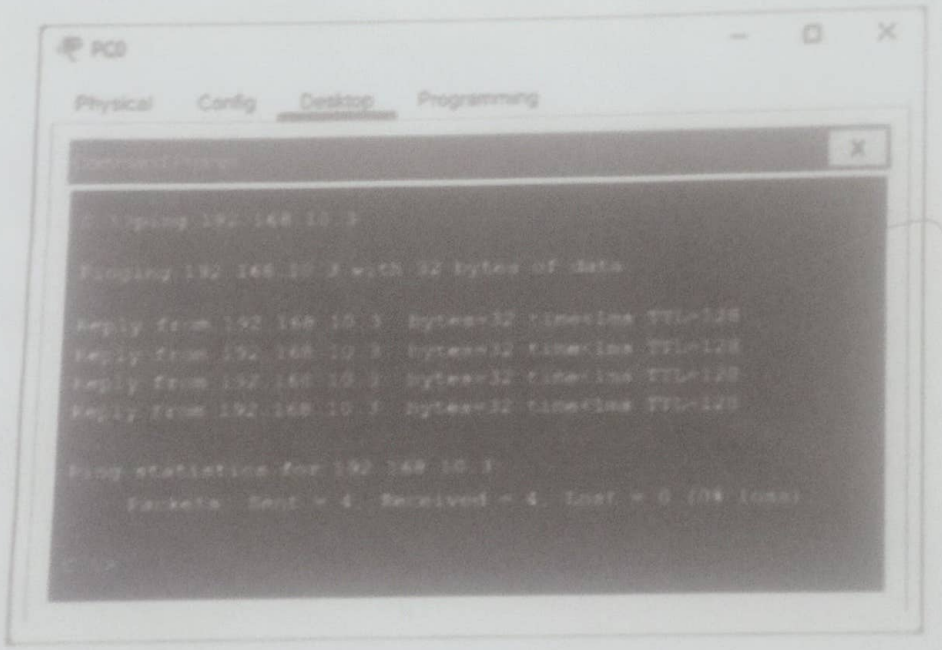
Aim:

To customize a Cisco switch by adding or removing network modules using cisco packet Tracer and verify the changes in the switch interfaces.

procedure:

1. Open cisco packet Tracer and wait for the workspace to load.
2. add a modular cisco switch (2960 or a supported model) to the workspace.
3. open the switch and select the physical tab to view the chassis and installed modules.
4. Turn off the switch using the power button.
5. Remove an existing module, if required, and insert a compatible network module into an empty expansion slot.
6. Turn on the switch and wait for it to boot completly.
7. configure pc0 with IP Address 192.168.1 and subnet 255.255.255.0


Cisco Packet Tracer
File Edit Options View Tools Extensions Help
Multilayer Switch0
PC0
PC1


PC0
Physical
Config
Desktop
Programming

Result :

Thus, the cisco switch was successfully customized by adding or removing network modules using cisco Packet tracer. The newly added interfaces were verified.

Ex.No: 11 Examine Network Address Translation

Date: 26/09/24 (NAT) using Cisco Packet Tracer

Aim:

To configure and examine Network Address Translation (NAT) using Cisco Packet Tracer and verify communication b/w a private network and a public network.

Procedure:

1. Open Cisco packet Tracer and create a new workspace.

2. Add the following devices: (Cisco 1941-2 1, 2960 switch -> 1, one PC, Server -> 1)

3 Connect the devices using copper straight - Through cables,

4. Configure the router's Inside Interface (Gig0/0) with IP Address 192.168.1.1 and subnet mask 255.255.255.0. Enable NAT Inside and activate the Interface.

5. Configure all.

6. Save the router configuration

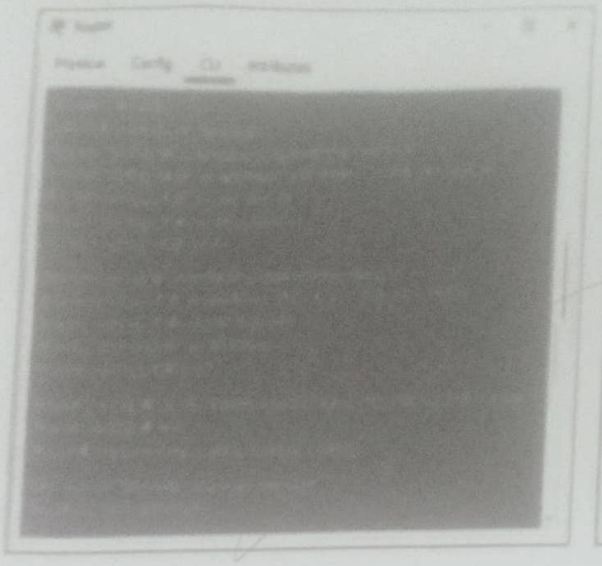
7. Configure PC0, configure the server

8. Execute all.

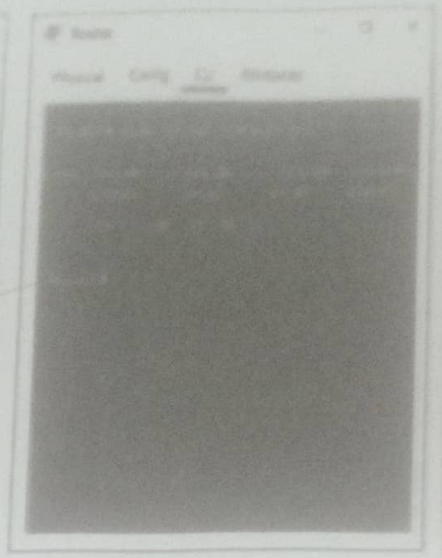
Result:

Thus the static Network Address Translation (NAT) was successfully configured using Cisco packet Tracer, The connectivity b/w private network.

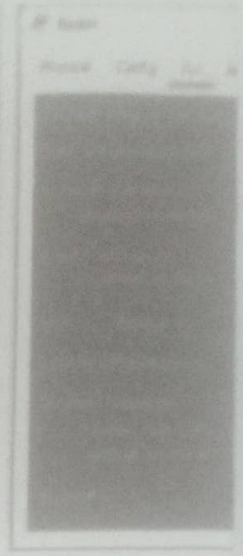
5. Router CLI – Interface Configuration (Partial)



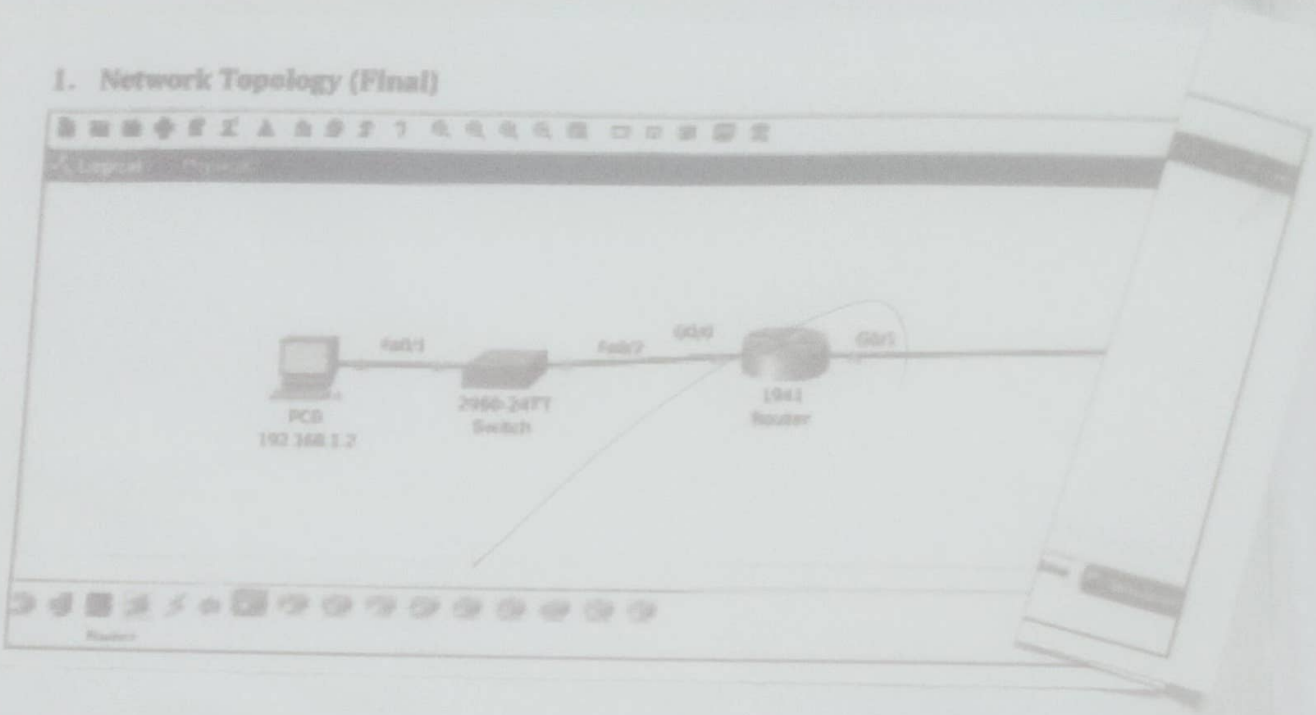
6. NAT Translation Table



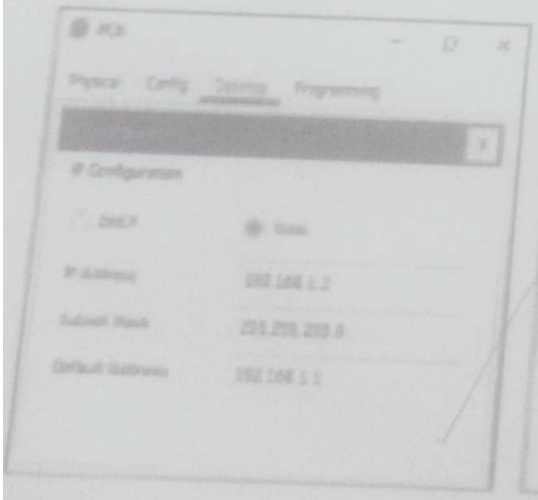
7. NAT Statistics



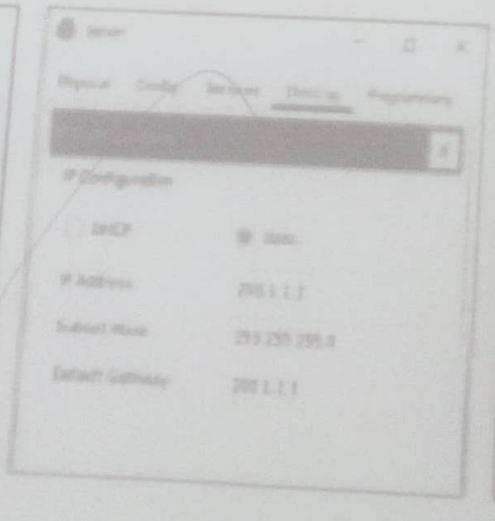
1. Network Topology (Final)



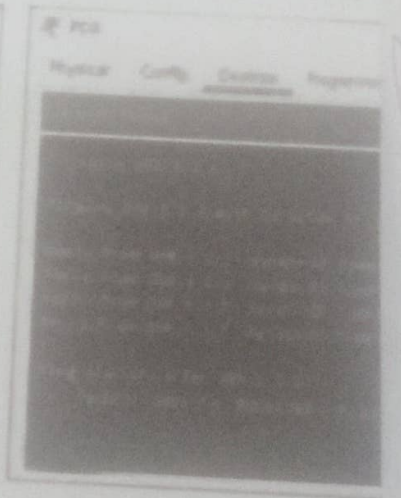
2. PC0 IP Configuration



3. Server IP Configuration



4. PC0 Command Prompt



Aim:

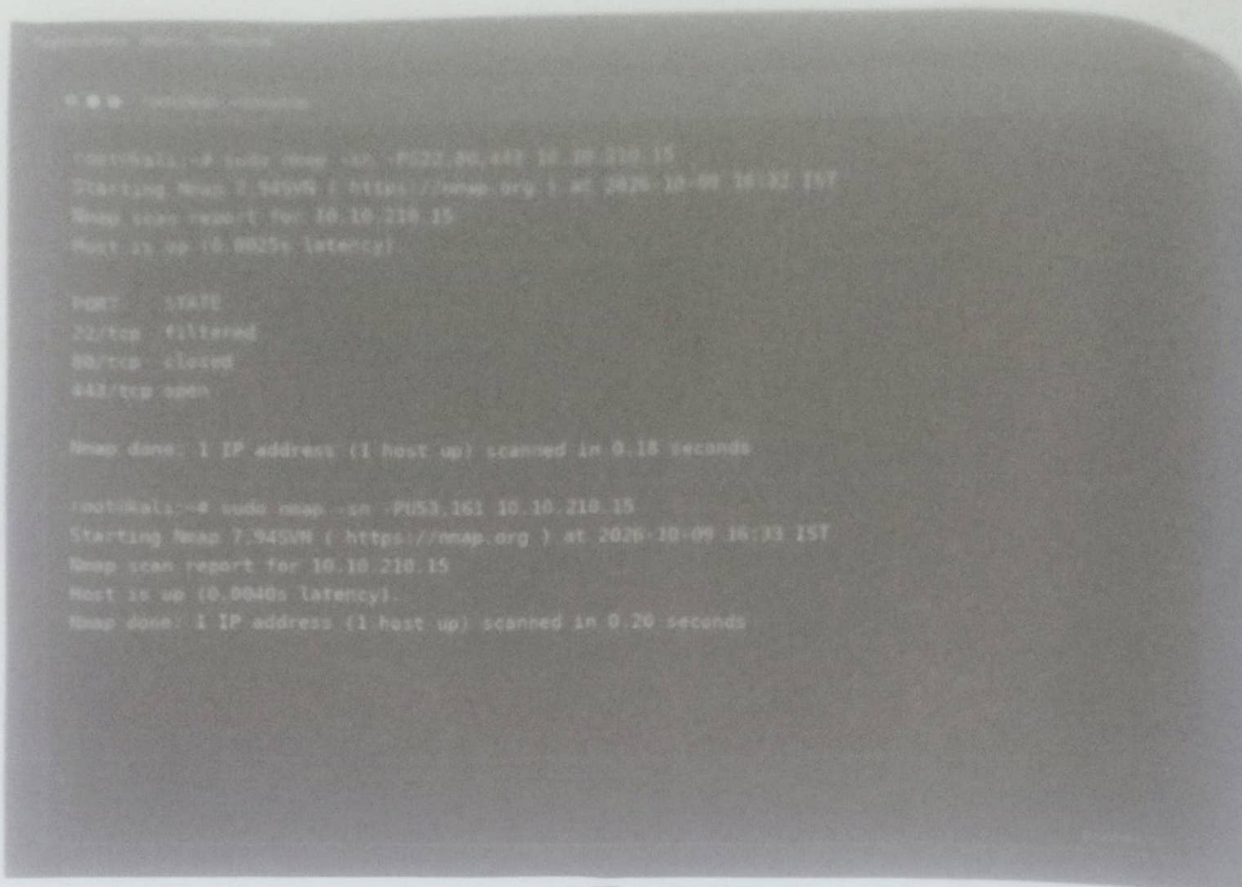
To discover live hosts in a network using Nmap by performing ARP scan, ICMP ping scan, TCP ping scan, and UDP ping scan on the TryHackMe platform.

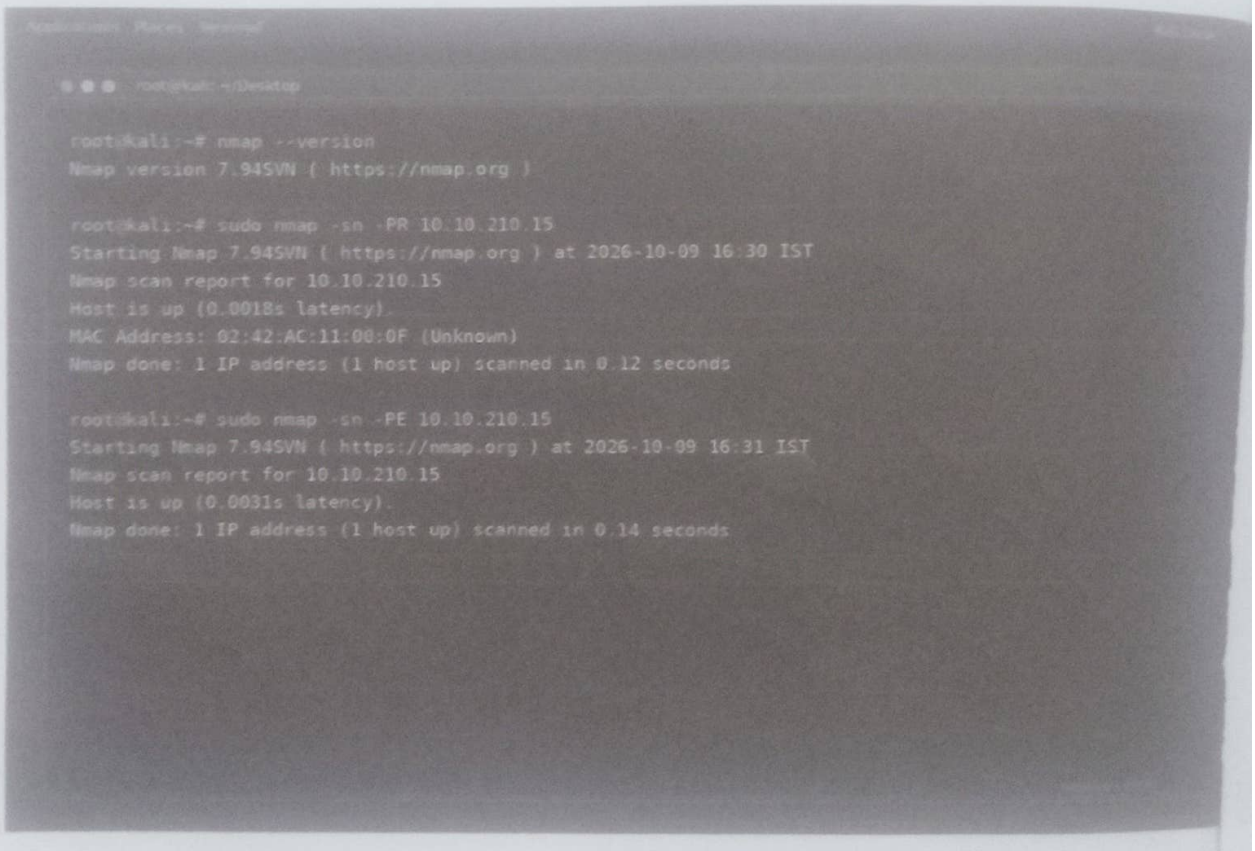
procedure:

1. login to TryHackMe
2. open required Room
3. Deploy the machine
4. Start the Attackbox
5. open terminal
6. perform ARP scan
7. perform ICMP ping scan
8. perform TCP ping scan
9. Perform UDP ping scan
10. Analyze the results.

Result:

Thus the host discovery was performed successfully using Nmap on the TryHackMe platform. ARP, ICMP, TCP, and UDP scans were executed.


root@kali:~# sudo nmap -sn -PS22,80,443 10.10.210.15
Starting Nmap 7.94SVN (https://nmap.org) at 2026-10-09 16:32 IST
Nmap scan report for 10.10.210.15
Host is up (0.0029s latency).
PORT STATE
22/tcp filtered
80/tcp closed
443/tcp open
Nmap done: 1 IP address (1 host up) scanned in 0.18 seconds
root@kali:~# sudo nmap -sn -PU53,161 10.10.210.15
Starting Nmap 7.94SVN (https://nmap.org) at 2026-10-09 16:33 IST
Nmap scan report for 10.10.210.15
Host is up (0.0040s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.20 seconds


root@kali:~# nmap --version
Nmap version 7.94SVN (https://nmap.org)
root@kali:~# sudo nmap -sn -PR 10.10.210.15
Starting Nmap 7.94SVN (https://nmap.org) at 2026-10-09 16:30 IST
Nmap scan report for 10.10.210.15
Host is up (0.0018s latency).
MAC Address: 02:42:AC:11:00:0F (Unknown)
Nmap done: 1 IP address (1 host up) scanned in 0.12 seconds
root@kali:~# sudo nmap -sn -PE 10.10.210.15
Starting Nmap 7.94SVN (https://nmap.org) at 2026-10-09 16:31 IST
Nmap scan report for 10.10.210.15
Host is up (0.0031s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.14 seconds

Ex:

Date : 01/08/25

Aim:

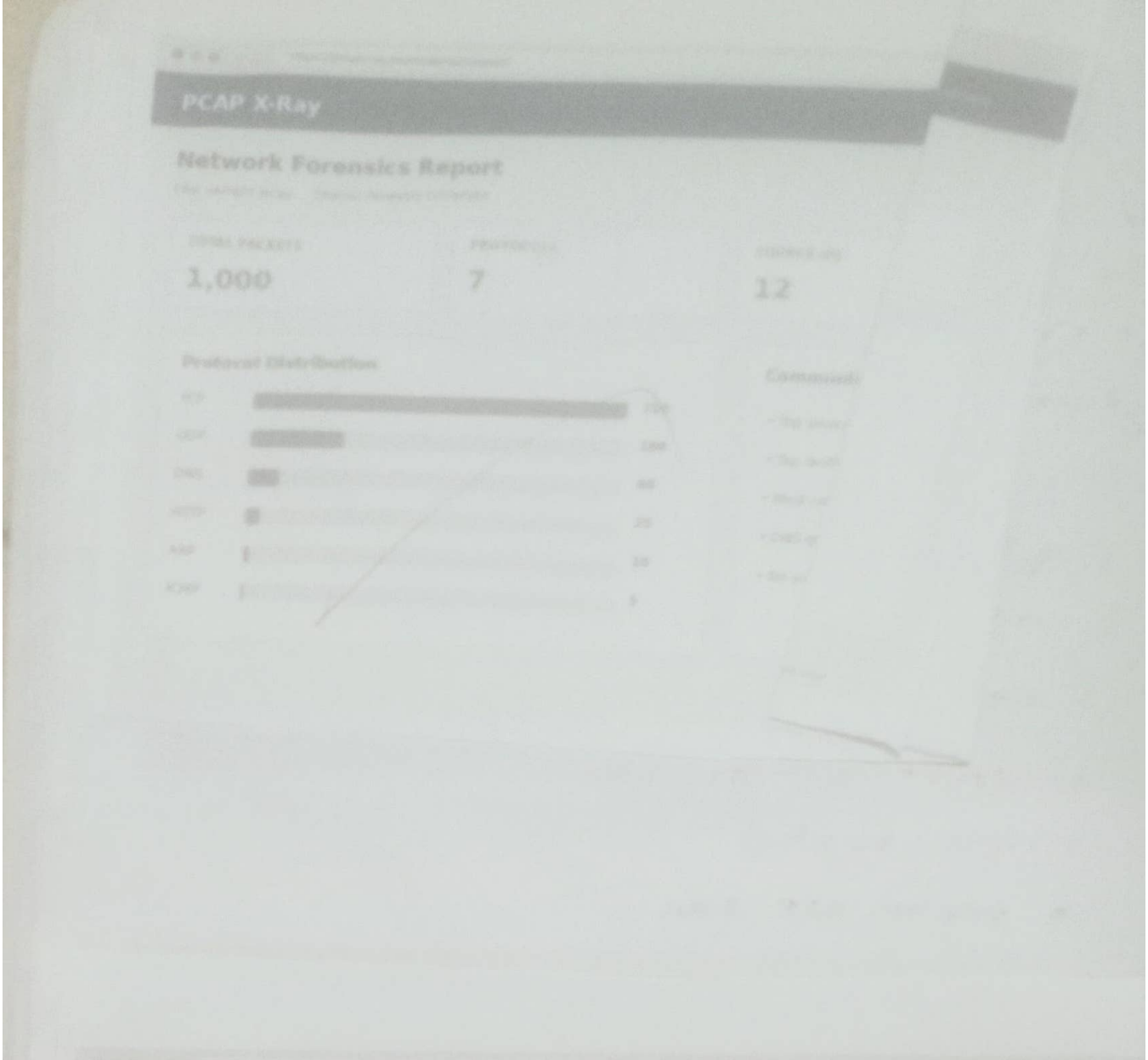
To perform network forensics by analyzing a packet capture (PCAP) file using the PCAP x-Ray tool to identify network protocols, suspicious activities, communication patterns, transmitted files, and security indicators.

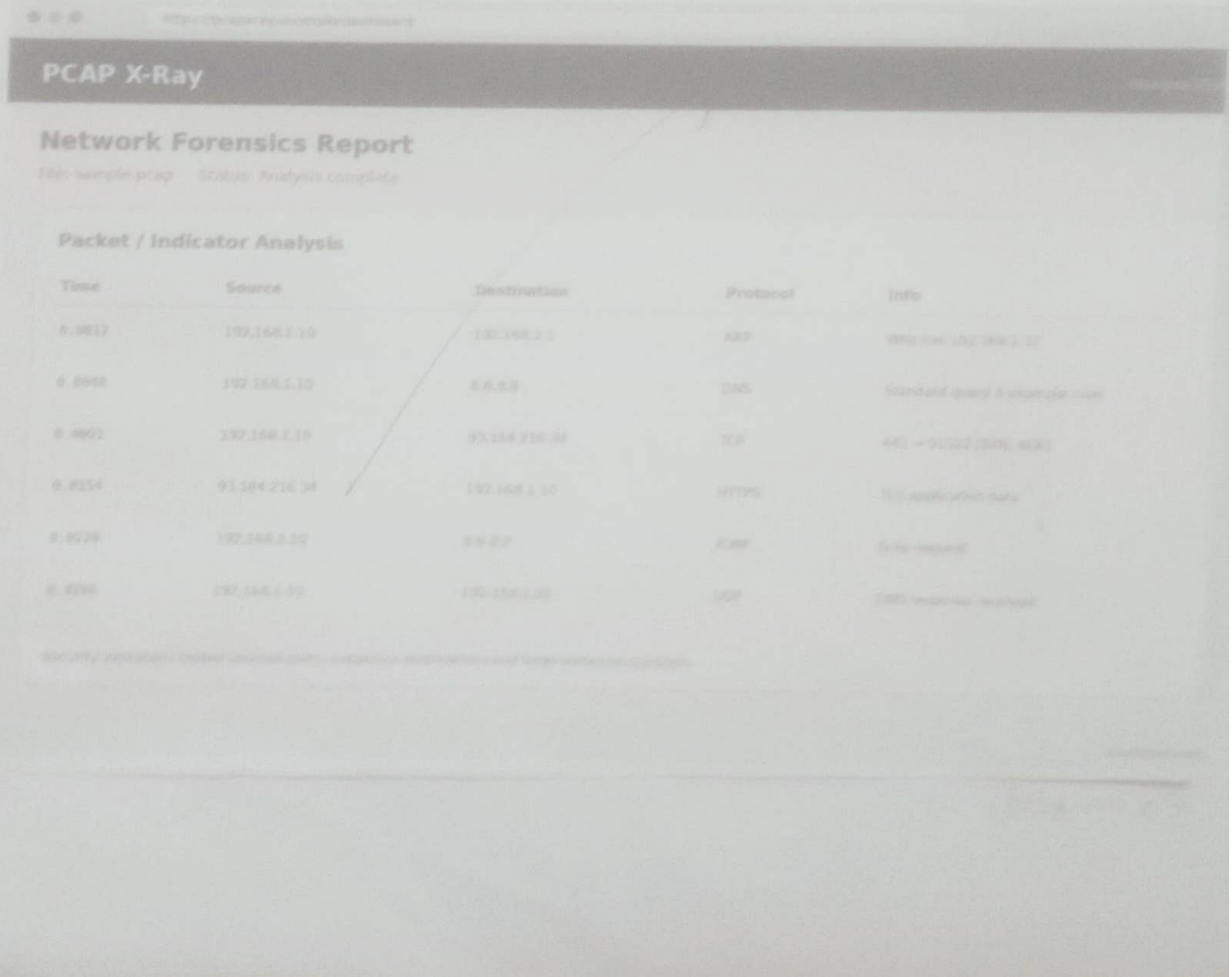
procedure :

1. obtain a PCAP file.
2. Capture your own traffic
3. Download a sample PCAP
4. open the PCAP x-Ray tool
5. Upload the PCAP file.
6. View the Dashboard after processing, the dashboard displays.
7. Analyze Protocol distribution
8. Examine Source and destination Hosts
9. finally Generate the Report.

Result :

Thus, network forensics was performed successfully using the PCAP x-Ray tool. The packet capture file was analyzed to identify network protocol, communication pattern.


PCAP X-Ray
Network Forensics Report
1,000
7
12
Protocol Distribution


PCAP X-Ray
Network Forensics Report
Packet / Indicator Analysis
Time
Source
Destination
Protocol
Info

Ex: No: 14

Date: 28/10/25

Analyze Network Traffic TCP, IP HTTP using Wireshark

Aim:

To capture, save, and analyze network traffic of TCP, UDP, IP, HTTP, ARP, DHCP, ICMP and DNS protocols using the wireshark network protocol analyzer.

Procedure:

Step 1: Install wireshark

Step 2: open wireshark

Step 3: Select the Network Interfaces

Step 4: Start capturing packets

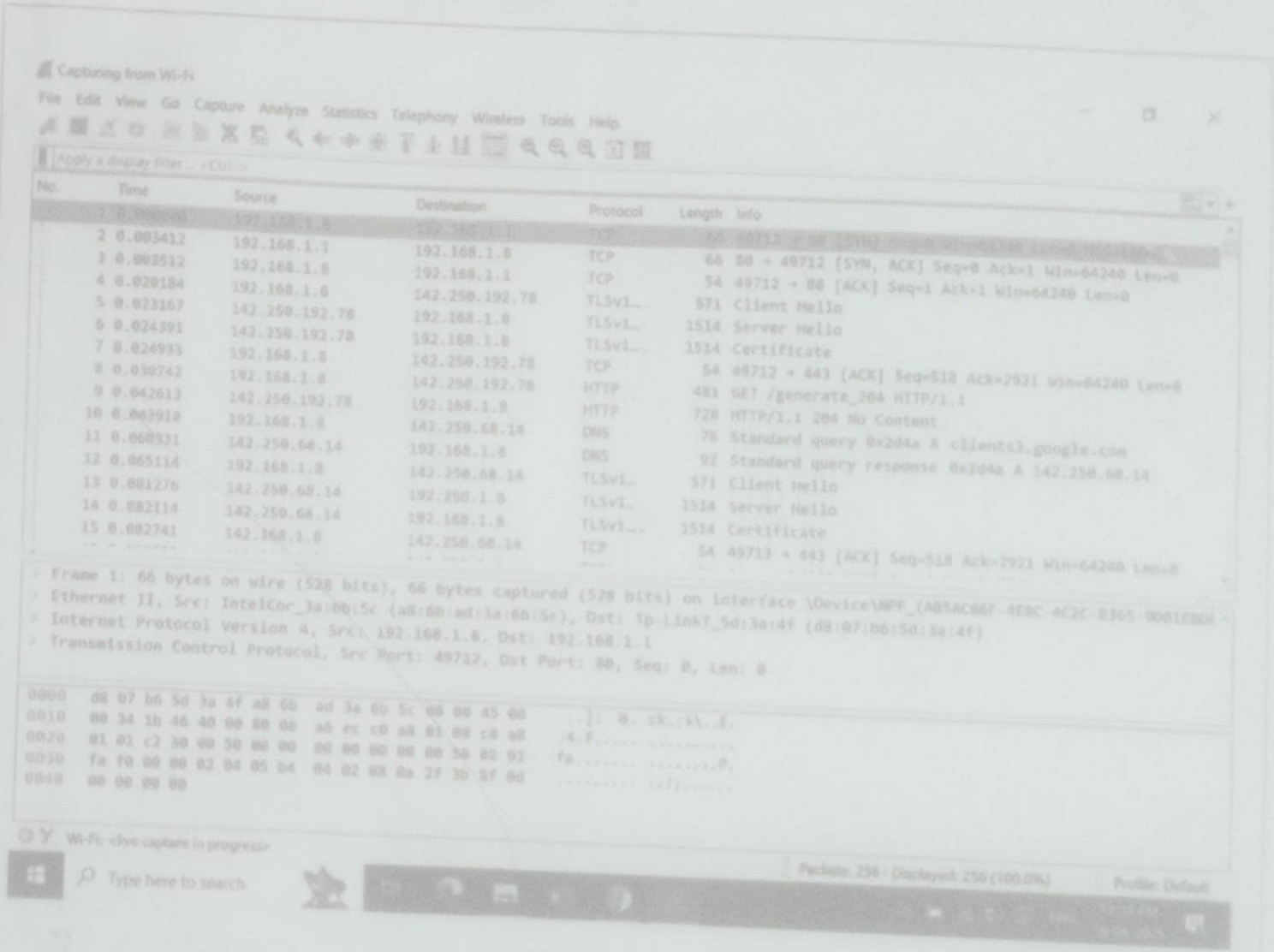
Step 5: Generate Network Traffic

* Capture ICMP packets
* Capture DNS packet
* capture HTTP packets
* capture TCP packets
* capture UDP packets
* capture ARP packets.
* capture DHCP packets.
* capture IP packets.

Step 6: Stop capture the file.

Result:

Network traffic for TCP, UDP, IP, HTTP, ARP, DHCP, ICMP, and DNS was captured and analyzed using wireshark. The captured packet were saved in .pcapng format for future analysis.


Capturing from Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000 192.168.1.8 192.168.1.1 TCP 66 49712 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1
2 0.003412 192.168.1.1 192.168.1.8 TCP 66 80 → 49712 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0
3 0.003512 192.168.1.8 192.168.1.1 TCP 54 49712 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
4 0.020184 192.168.1.8 142.250.192.78 TLSv1… 571 Client Hello
5 0.023167 142.250.192.78 192.168.1.8 TLSv1… 1514 Server Hello
6 0.024391 142.250.192.78 192.168.1.8 TLSv1… 1514 Certificate
7 0.024933 192.168.1.8 142.250.192.78 TCP 54 49712 → 443 [ACK] Seq=518 Ack=2921 Win=64240 Len=0
8 0.030742 192.168.1.8 142.250.192.78 HTTP 481 GET /generate_204 HTTP/1.1
9 0.042613 142.250.192.78 192.168.1.8 HTTP 728 HTTP/1.1 204 No Content
10 0.043918 192.168.1.8 142.250.68.14 DNS 76 Standard query 0x2d4a A clients3.google.com
11 0.060531 142.250.68.14 192.168.1.8 DNS 92 Standard query response 0x2d4a A 142.250.68.14
12 0.065114 192.168.1.8 142.250.68.14 TLSv1… 571 Client Hello
13 0.081276 142.250.68.14 192.250.1.8 TLSv1… 1514 Server Hello
14 0.082114 142.250.68.14 192.168.1.8 TLSv1… 1514 Certificate
15 0.082741 142.168.1.8 142.250.68.14 TCP 54 49713 → 443 [ACK] Seq=518 Ack=2921 Win=64240 Len=0
Frame 1: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{A85AC86F-4E8C-4C2C-8365-9D61EB0A…
Ethernet II, Src: IntelCor_3a:6b:5c (a8:6b:ad:3a:6b:5c), Dst: Tp-LinkT_5d:3a:4f (d8:07:b6:5d:3a:4f)
Internet Protocol Version 4, Src: 192.168.1.8, Dst: 192.168.1.1
Transmission Control Protocol, Src Port: 49712, Dst Port: 80, Seq: 0, Len: 0
0000 d8 07 b6 5d 3a 4f a8 6b ad 3a 6b 5c 08 00 45 00 ...]:O. :k\..E.
0010 00 34 1b 46 40 00 80 06 a6 ec c0 a8 01 08 c0 a8 .4.F@... ....
0020 01 01 c2 30 00 50 00 00 00 00 00 00 00 00 80 02 ...0.P.. ....
0030 fa f0 00 00 02 04 05 b4 04 02 08 0a 2f 3b 9f 0d ..../;..
0040 00 00 00 00
Wi-Fi: <live capture in progress>
Packets: 256 · Displayed: 256 (100.0%)
Profile: Default
Type here to search

Aim: To Analyze the different type of web servers using webalizer tool

Aim:

To install, configure, run the webalizer and to analyze web server log files and generate statistical reports for different types of web servers.

procedure:

Step 1 : Update the system
step 2 : Install Apache web server
Step 3 : Verify Apache server
Step 4 : Generate web traffic
Step 5 : Locate the Apache log file
step 6 : Install webalizer
Step 7 : Configure webalizer
Step 8 : Create the output directory
step 9 : Generate the Report
Step 10 : View the Generated Report
Step 11 : Analyze the Reports

Result:

The webalizer tool was installed and configured to analyze web server log files. statistical and graphical reports containing visitors details page views, hits, HTTP status codes, referring sites, browser statistics generated & analyzed successfully.

EXPERIMENT 15 – WEBALIZER SERVER LOG ANALYSIS

Sample web server statistics and HTTP response report

Webalizer Web Server Statistics

Usage Statistics for localhost

Hits	Files	Pages	Visits	Sites
1,248	986	412	173	48

Monthly History

Month	Hits	Files	Pages	Visits	Sites
Oct 2026	1,248	986	412	173	48
Sep 2026	1,109	874	365	142	41
Aug 2026	987	771	321	128	39

Daily Usage (October 2026)



Webalizer Web Server Statistics

Usage Statistics for localhost

Hits	Files	Pages	Visits	Sites
1,248	986	412	173	48

Daily Statistics and Request Summary

Top Requested Pages

/	312
/index.html	244
/about.html	121
/contact.html	94
/images/logo.png	83
/products.html	71

HTTP Response Codes

